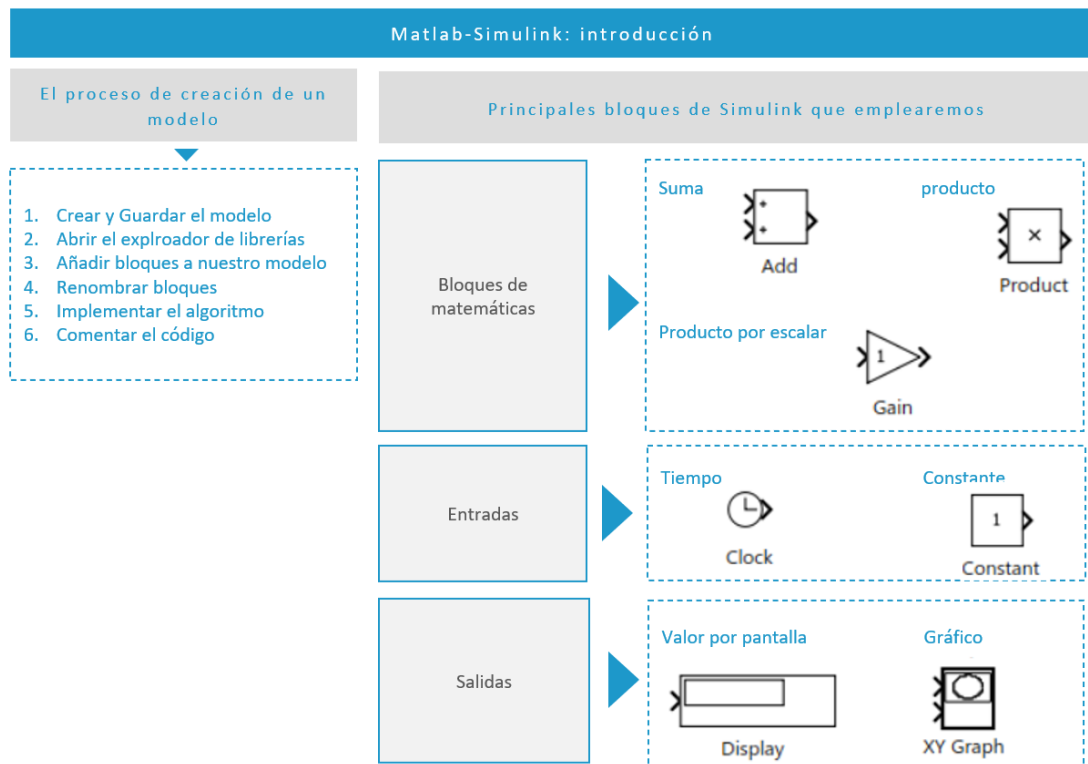


Modelado de Sistemas Dinámicos

Matlab - Simulink: introducción

Índice

Esquema.	2
Ideas clave	3
2.1 Introducción y objetivos	3
2.2 Simulink como entorno de modelado y simulación	4
2.3 Los principales elementos de Simulink.	9
2.4 El proceso de creación de un modelo	13
2.5 Tocando Simulink: “Primeros códigos en Simulink”	15



2.1 Introducción y objetivos

El modelado y la simulación pueden tener diferentes aspectos de gran complejidad. Para simplificar el proceso podemos utilizar software que nos permita realizar el proceso de modelado de forma más simple.

En este curso nos vamos a centrar en el modelado mediante Simulink. Simulink es un paquete para Matlab que nos permite trabajar y modelar ecuaciones mediante un sistema gráfico. Decimos que Simulink es un entorno de programación gráfico. Por entorno de programación entendemos un software que nos permite dar instrucciones a un ordenador para obtener un cierto resultado esperado. Ya sea una fórmula sencilla como la resolución de una ecuación de segundo grado dados los tres coeficientes, o un problema complejo que requiera de un método numérico para su resolución. La gran diferencia con los entornos de programación clásicos viene dada por el adjetivo gráfico. Esta propiedad hace que la lectura de un código en matlab se simplifique al igual que podemos pedir la cuenta a un camarero en un bar haciendo un pequeño gesto con la mano.

Partiendo de esta premisa, el código en Simulink pretende ser comprensible y fácil de explicar incluso para alguien que no entienda de programación. La figura 1 nos muestra un código en Simulink. Probablemente, uno de los códigos más sencillos que podemos pensar. Un usuario atento, enseguida verá qué realiza el programa: toma dos valores, calcula su suma y nos devuelve el resultado. Esto, lo detectará sin conocer el lenguaje y, probablemente lo pueda hacer sin conocer ningún otro lenguaje de programación. Algo que sería raro en un lenguaje convencional.

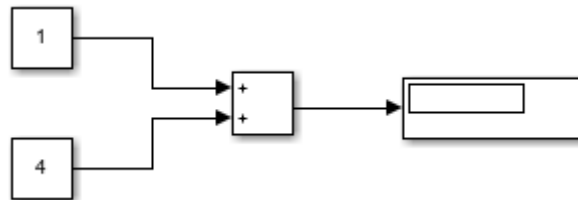


Figura 1: Ejemplo de un código en matlab para calcular la suma de dos términos.

A lo largo del curso, desarrollaremos diferentes conceptos relacionados con el mode-

lado y los combinaremos con los conceptos necesarios de Simulink. En este tema nos vamos a fijar como objetivos:

- ▶ Saber iniciar Simulink.
- ▶ Conocer las operaciones básicas en Simulink.
- ▶ Saber realizar un primer programa en Simulink.

2.2 Simulink como entorno de modelado y simulación

En esta sección vamos a seguir los pasos necesarios para realizar un primer programa en Simulink. Vamos a realizar el código que se muestra en la figura 1 que realiza la suma de dos números. Podemos pensar que estamos modelando el problema: *Tenemos dos depósitos de agua, uno de 1 litro y otro de 4 litros. ¿Cuál es el volumen que podemos almacenar?*. Evidentemente, este problema no requiere de ningún programa para resolverlo. Sin embargo, este ejemplo nos permitirá ver rápidamente las ideas básicas de Simulink.

El primer problema con el que podemos encontrarnos es la instalación de Simulink. Al instalar Matlab Simulink viene como paquete por defecto y se instala automáticamente. Si no fuera así, el vídeo “El gestor de *addons*: Instalando Simulink” podrás aprender como instalarlo. Además, también explicamos como podemos instalar otros paquetes que nos puedan interesar.



Accede al vídeo: El gestor de *addons*: instalando Simulink

el enlace a la píldora se adjunta posteriormente en la etapa de postproducción

Una vez tenemos instalado Simulink, debemos abrir Matlab y pulsar sobre el botón Simulink, tal y como se muestra en la Figura 2. Con esto se nos abrirá la ventana de inicio de Simulink. Deberemos situarnos encima de *blank model* y pulsar sobre el botón *create model* que nos aparece.

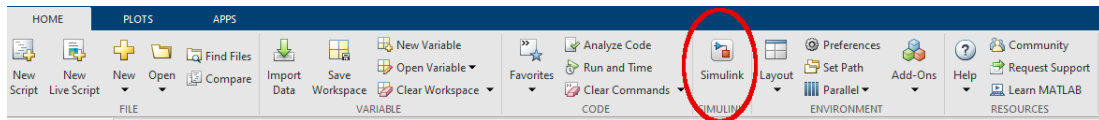


Figura 2: Lista de comandos disponibles en Matlab, destacando el botón de Simulink.

Se nos abrirá una nueva ventana como la que se muestra en la Figura 3. En esta ventana encontraremos nuestro lienzo en blanco para realizar plantear un modelo. Marcado con un 1 tenemos el espacio destinado a la elaboración del modelo. En este espacio añadiremos las diferentes piezas y componentes que necesitemos. El botón *library Browser* (2) nos permite abrir la ventana de herramientas y elementos de Simulink, esta será nuestra pieza clave a lo largo de todo el proceso. Finalmente, una vez tengamos completado el modelo, podremos simularlo usando las herramientas que aparecen en la sección 3.

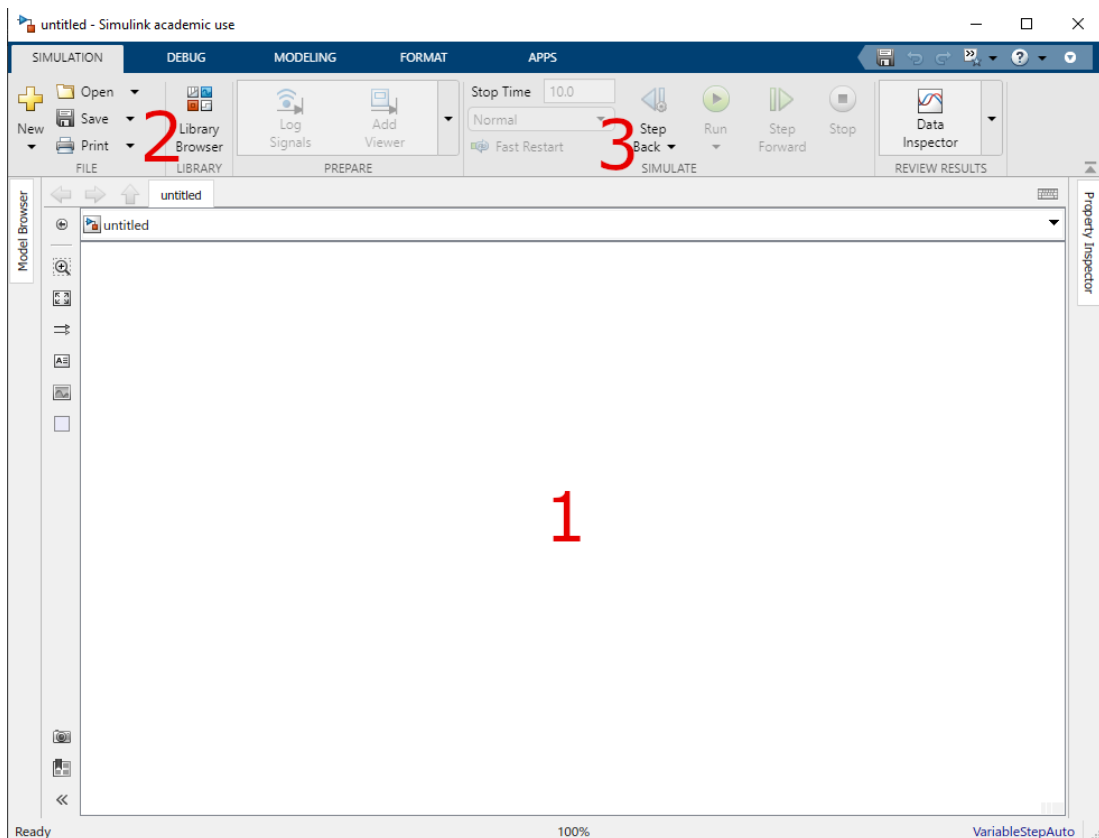


Figura 3: Entorno básico de programación de Simulink. 1) espacio para el modelo, 2) botón para abrir las herramientas de modelado. 3) espacio para gestionar la simulación.

Vamos a prescindir de la mayor parte de las opciones disponibles en este primer ejemplo. Por ahora, será suficiente con abrir el *Library Browser*. Esto nos abrirá una nueva

ventana (ver figura 4). Esta ventana nos permitirá seleccionar aquellas herramientas que deseemos incorporar a nuestro modelo. En la parte izquierda se nos mostrará el listado de paquetes que tenemos instalados. En la instalación básica tendremos únicamente Simulink. Dependiendo de los paquetes que tengamos instalados, se añadirán otras opciones desplegadas. La navegación se realiza de forma intuitiva siguiendo la estructura de navegación por carpetas de los sistemas operativos actuales. Si desplegamos la opción de Simulink, podemos acceder a las diferentes subcarpetas que forman parte del paquete esencial. En la sección 2.3 describiremos los elementos que podemos encontrar.

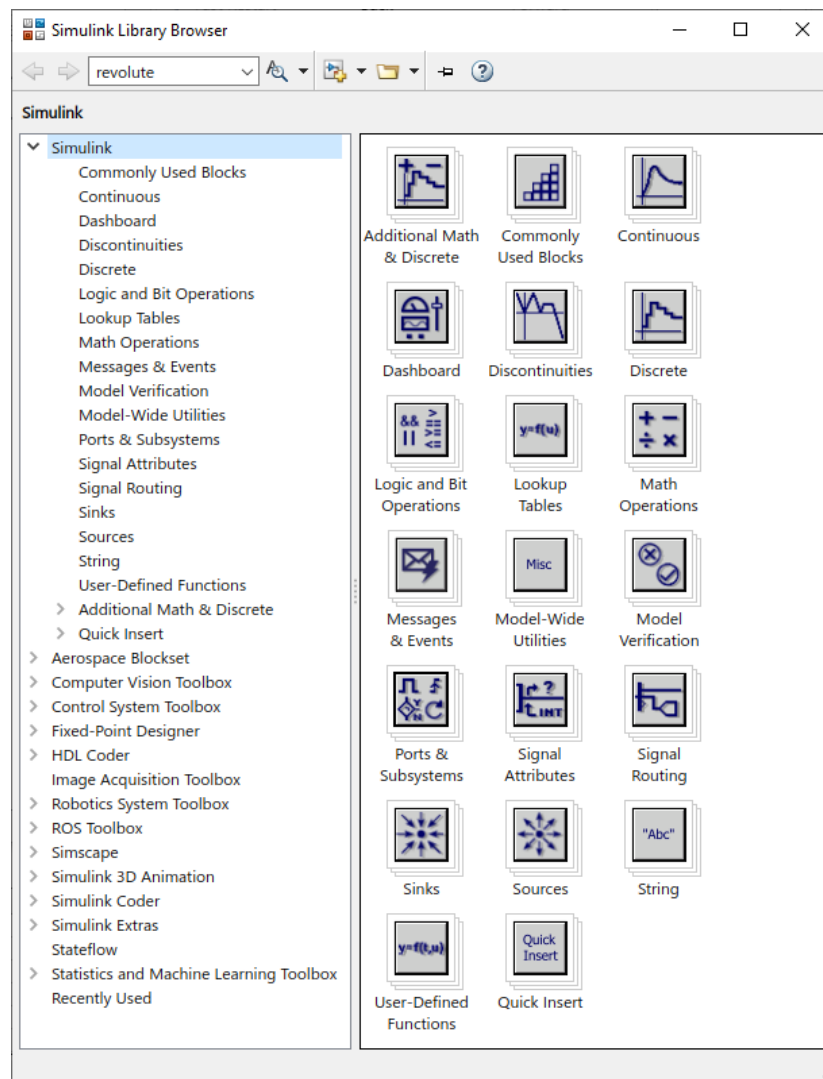


Figura 4: Explorador de librerías y elementos de Simulink.

Por ahora, nos vamos a centrar en tres de estos bloques. El bloque de **fuentes** (*sources*) que nos permitirá introducir información a nuestro sistema, el bloque de **salidas** (*sinks*) que nos permitirá leer y/o obtener la información de nuestro modelo y, finalmente, el

bloque de **operaciones matemáticas** (*Math Operations*) en el que encontraremos las operaciones matemáticas necesarias para nuestro modelo simple.

Vamos a necesitar los bloques siguientes:

- ▶ Dos bloques de fuente constante. Los podemos encontrar en la ruta Simulink→Sources→Constant.
- ▶ Un bloque de salida de información tipo *display*. Lo podemos encontrar en la ruta Simulink→Sinks→Display.
- ▶ Un bloque de suma que podemos encontrar en la ruta Simulink→Math Operations→Add.

Cuando encontremos los diferentes bloques, deberemos incluirlos en nuestro lienzo de trabajo arrastrándolos. Llegados a este punto, tendremos algo parecido a la imagen de la figura 5.

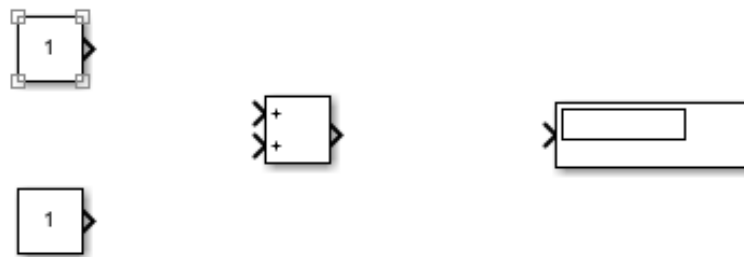


Figura 5: Distribución de los elementos necesarios para el ejemplo inicial después de introducirlos.

Con el fin de clarificar los contenidos de nuestro modelo, vamos a asignar nombres a los dos depósitos. Para ello, pulsamos sobre el elemento constante que hemos introducido. En la parte inferior del mismo, nos aparecerá el nombre del elemento. Podemos modificar el nombre por el nombre deseado. Si dejamos el bloque con su nombre por defecto, no aparecerá pero si lo modificamos, quedará reflejado el nombre. Esto nos ayudará a comprender qué hace el modelo y detectar fácilmente los posibles errores que cometamos.

El siguiente paso, será configurar los elementos. Únicamente debemos configurar el segundo de los depósitos. Para ello, deberemos realizar un doble *clíc* en el elemento que representa el segundo depósito. Se nos abrirá la ventana de configuración del

elemento. Cada elemento tiene asociadas distintas propiedades. En el caso del elemento *constant* podemos configurar su valor cambiando el número debajo de la etiqueta *Constant Value*:. Lo actualizamos a 4 que es el valor deseado y pulsamos *ok*. Al volver a nuestro modelo, nos aparecerá el nuevo valor.

El siguiente paso será conectar los diferentes elementos del modelo. Para ello, deberemos unir las entradas y salidas de los diferentes bloques. Cada bloque puede tener una o varias entradas y/o salidas. En nuestro modelo tenemos: el bloque constante que tiene una única salida, el bloque de suma que tiene dos entradas y una salida y el bloque de mostrar el valor por pantalla que tiene una única entrada.

Todos los puertos de entrada y salida deben acabar conectados al finalizar el modelo. Para conectar un puerto podemos arrastrar la flecha del puerto salida del elemento deseado hasta el puerto de entrada del elemento correspondiente. Vamos a enlazar los elementos de la forma natural. Cada depósito a un puerto de entrada de la suma, y el puerto de salida de la suma al elemento *display*.

Con esto completamos nuestro primer modelo en Simulink. Ahora solo nos queda realizar la simulación del modelo para obtener los resultados. Para ello, debemos pulsar el botón *run* en el bloque de *simulate* (3) de la figura 3. El resultado final se muestra en la figura 6. En el podemos ver como el bloque de muestra de resultados nos indica el resultado esperado.

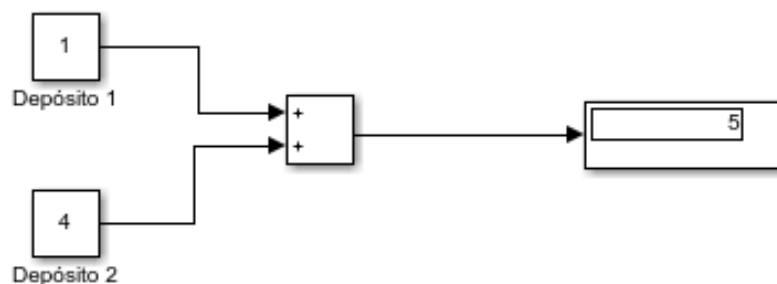


Figura 6: Modelo y resultado final.

2.3 Los principales elementos de Simulink

En esta sección veremos los apartados y los tipos de instrucción elementales que podemos encontrar en Simulink. A lo largo del curso, profundizaremos en los elementos y opciones más importantes e introduciremos algunos avanzados que puedan resultar de interés.

Bloques matemáticos

En esta pestaña encontraremos todos los bloques relacionados con operaciones matemáticas. Ya sean operaciones elementales como la suma y la resta (bloque Add), el producto y la división (bloque Product y su variante Divide), el cálculo de funciones elementales (bloque Math Function) o el valor absoluto (bloque Abs).

También podremos utilizar funciones vectoriales como el producto vectorial (bloque Dot Product) matriciales como la concatenación de matrices. Las funciones de suma y producto funcionan de la forma usual de Matlab. Así pues, podemos tener matrices introducidas en nuestro código y operarlas normalmente.

Podemos ver algunos ejemplos de bloques en la Figura 7

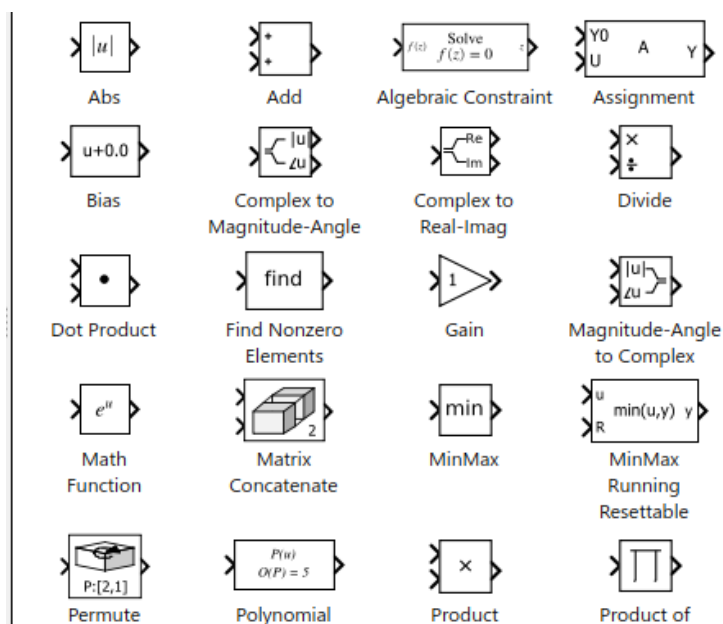


Figura 7: Bloques relacionados con operaciones matemáticas de Simulink

Bloques para sistemas dinámicos.

Tenemos dos pestañas que nos permitirán introducir dinámica en un sistema según este sea un sistema continuo o discreto. La primera de las dos pestañas es la pestaña de bloques de dinámica continua (*continuous*), ver la Figura 8. En ella podremos introducir diferentes valores para modelar sistemas de ecuaciones que vengan definidos por ecuaciones diferenciales. El sistema nos permite añadir un integrador numéricos a nuestro modelo para poder implementar la ecuación diferencial (bloques *integrator* y *Integrator second order*). También introduce otras funcionalidades como el cálculo de derivas o la inclusión de funciones de transferencia o descripciones de sistemas mediante el par estado-espacio.

Un conjunto que puede resultar interesante es el uso de controladores PID (proporcionales, integrales y diferenciales) este tipo de controlador se utiliza ampliamente en sistemas reales para aproximar de forma correcta un cierto valor.

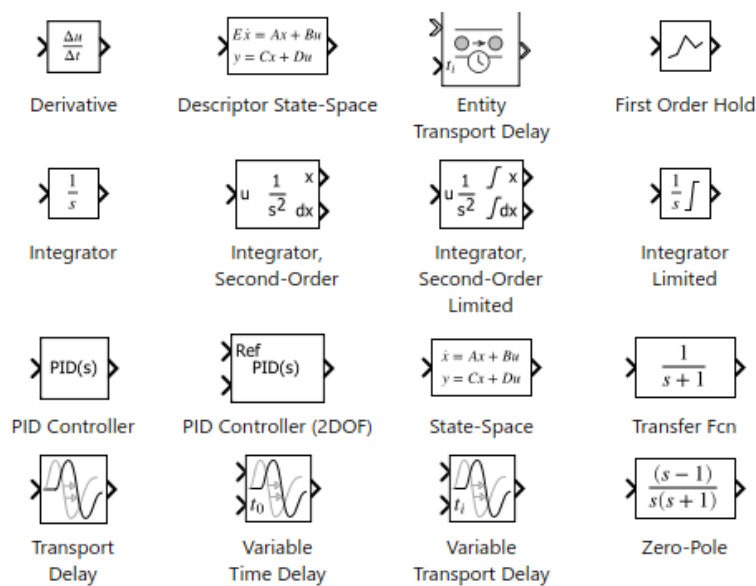


Figura 8: Bloques para trabajar con sistemas continuos en Simulink

La segunda pestaña que nos permitirá relizar momdelos de sistemas dinámicos es la pestaña de los modelos dinámicos discretos (*discrete*), ver Figura 9. En esta ventana encontraremos las herramientas para realizar avances en tiempo discreto (bloque *Delay*) así como herramientas para modelar sistemas mediante funciones de transferencia discretas o funciones de zeros y polos.

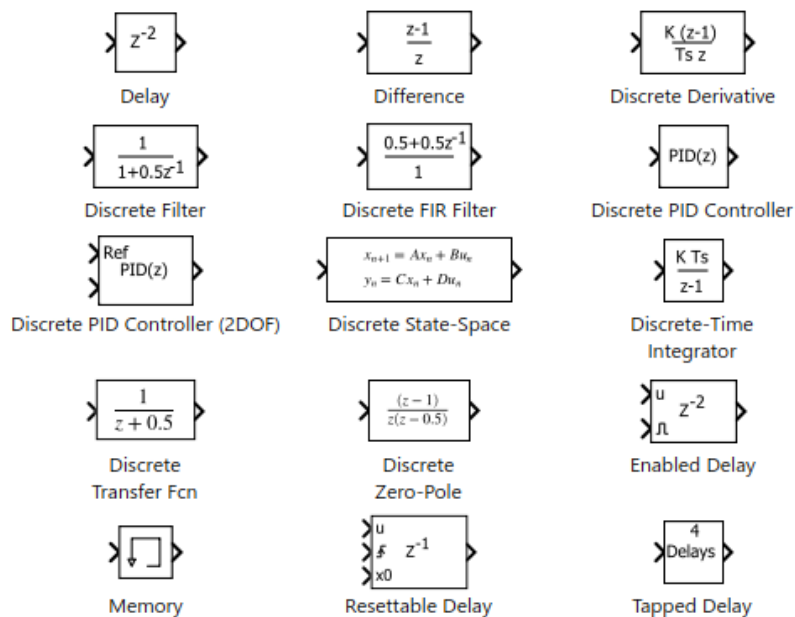


Figura 9: Bloques para trabajar en sistemas discretos en Simulink

Bloques de entrada y salida

Los bloques de entrada y salida de un modelo nos permiten introducir y leer resultados de un sistema. Bajo la pestaña de bloques de entrada (*source*, del inglés fuente) podremos escoger multitud de bloques para introducir señales a nuestro modelo. Hemos visto el uso de la señal constante (Constant), pero también podemos introducir señales sinusoidales (Sine Wave), señales escalón (Step), señales aleatorias (Uniform Random Number) o escoger variables del *Workspace* de Matlab (From Workspace o de un archivo (From File)). También tenemos la opción de incluir el bloque *clock* que nos permite iniciar una señal con el tiempo interno de la simulación.

Dos bloques importantes que veremos en temas posteriores son el bloque de tierra (ground) que nos permite indicar que una señal no queda conectada y el bloque de entrada a un subsistema (in) que nos permite pasar variables a un subsistema para clarificar el estado. Tenemos que tener en cuenta, que todas las señales deben iniciar y terminar en un bloque. Nunca podemos dejar un puerto de entrada sin una señal, ni una señal sin puerto de entrada.

En cuanto a los bloques de salida (bajo la pestaña *sinks*, del inglés sumidero) son aquellos bloques que nos permiten obtener resultados por pantalla (Display) o exportarlos a otro subsistema (out), fichero (To file) o variable (To Workspace).

También tendremos la opción de generar gráficos mediante los bloques *scope* y *XY*

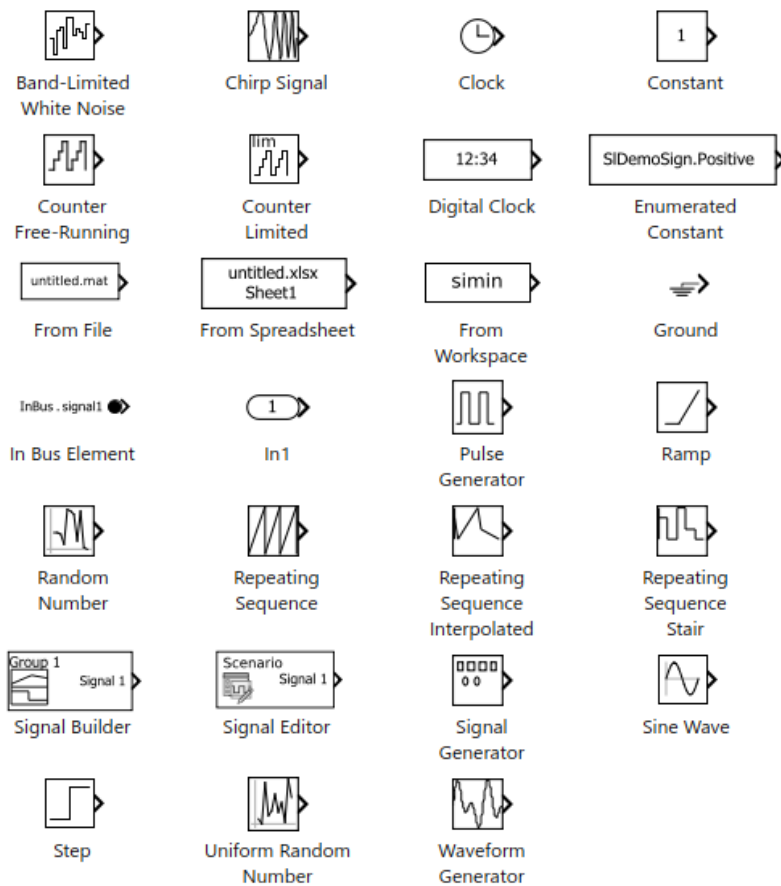


Figura 10: Bloques para la inserción de datos en Simulink

Graph.

Al igual que sucedía con las entradas, tenemos una salida a nada bajo el bloque Terminator. Esto nos permitirá finalizar un hilo que no nos interese de un bloque. Sucederá en situaciones raras, pero puede ocurrir cuando el bloque en cuestión tenga más de una salida y únicamente estemos interesados en una de ellas.

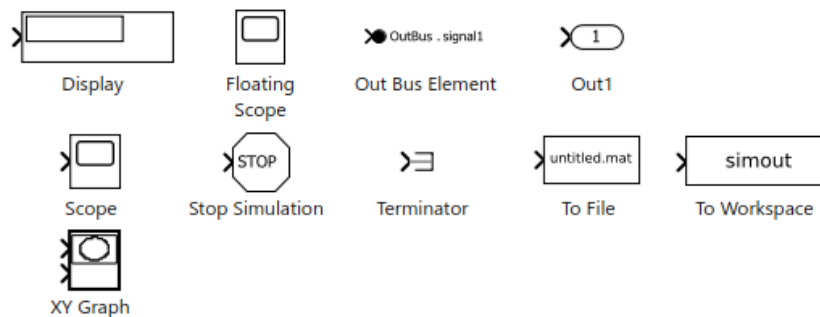


Figura 11: Bloques para la lectura de datos en Simulink

Bloques de uso habitual

La última pestaña que vamos a describir es la pestaña de bloques de uso habitual. En esta pestaña vamos a encontrarnos la mayoría de los bloques ya ya hemos visto anteriormente. Hay algunos bloques adicionales como los bloques *Demux* y *Mux* que permiten unir señales, el bloque *Switch* que actúa a modo de condicional if-else y el bloque *Subsystem* que nos permitirá introducir un subsistema con entradas y salidas. También tenemos bloques clásicos de la teoría de diagramas de bloques como el bloque *Gain* que permite multiplicar por un escalar y el bloque *Sum* que realiza la suma de las dos señales.

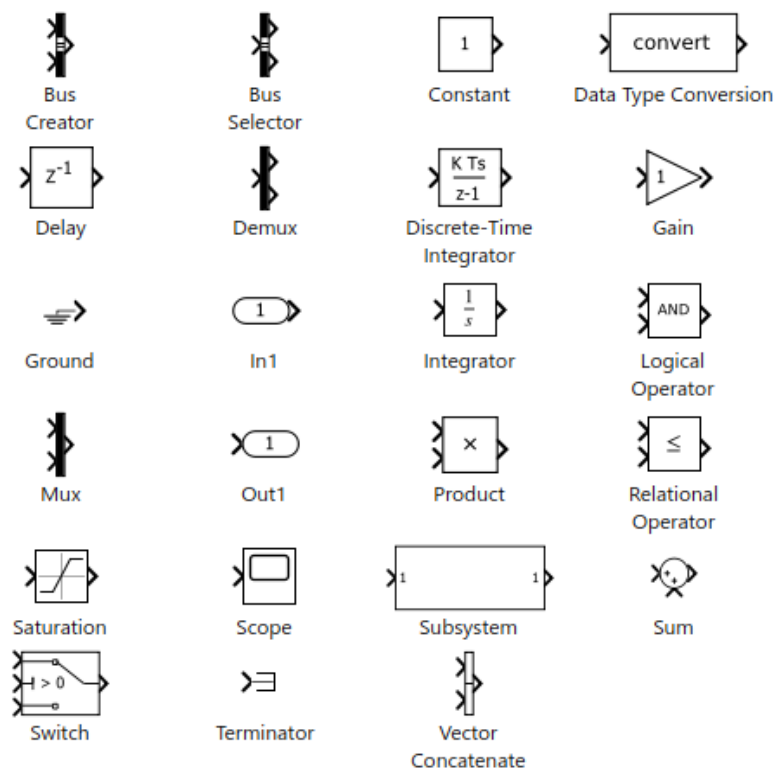


Figura 12: Atajo de bloques comunes en Simulink

2.4 El proceso de creación de un modelo

En la siguiente sección seguiremos los pasos necesarios para la implementación de un modelo en Simulink. Vamos a suponer que tenemos un modelo ya diseñado y queremos implementarlo. A partir de este punto deberemos:

1. Crear y guardar el modelo. Ya hemos visto como crear un nuevo partiendo desde Matlab. Partiremos de la ruta Simulink→ Create Blank Model. Una buena costumbre consiste en guardar el modelo nada más empezar. Podremos elegir entre dos extensiones:
 - ▶ .mdl: es la extensión que utilizan las versiones de Matlab anteriores a la R2012a. Se introduce por motivos de retrocompatibilidad. Todavía hoy en día muchos equipos siguen empleandola puesto que los modelos están diseñados en este sistema de archivos.
 - ▶ .slx: es la extensión que se incorporó a partir de la versión R2012a. Consiste en un archivo comprimido que contiene el resto de valores.
2. Abrir la el explorador de librerías de Simulink. Este siempre será nuestro segundo paso para trabajar con Simulink puesto que aquí encontraremos todas nuestras herramientas o bloques.
3. Añadir bloques a nuestro modelo. Podemos añadir un nuevo bloque a través de los siguientes métodos:
 - ▶ Arrastrando el bloque a nuestro modelo.
 - ▶ Copiando y pegando el bloque (los atajos típicos Ctrl+C, Ctrl+V también funcionan. También podemos arrastrar un bloque con el botón derecho para crear una copia del mismo.
 - ▶ Pulsando con el botón izquierdo en el elemento que queremos introducir y después en Add to model NombreDeMiModelo.
 - ▶ Usando el atajo de teclado Ctrl+I.

Si realizamos una doble pulsación en un elemento nos aparecerá la ventana de configuración. En ella podremos obtener más información sobre las funcionalidades del modelo.

Una buena práctica cuando trabajamos en Simulink consiste en tener los bloques lo mejor ordenados posible. Así la lectura del documento será más sencilla e intuitiva.
4. Renombrar los bloques. En nuestro ejemplo inicial hemos visto que si pulsamos un bloque en nuestro modelo nos aparece el nombre resaltado y podemos cambiarlo. Esta práctica de nuevo nos ayudará a mantener el código fácil de entender y limpio. Recuerda que no podemos tener dos bloques con el mismo nombre.
5. Implementar el algoritmo. En este paso tendremos que realizar las conexiones necesarias para implementar el algoritmo. También deberemos dar formato a los diferentes bloques introduciendo sus propiedades.

Podemos unir dos bloques arrastrando la punta de flecha de entrada del primero hasta la flecha de destino. Si dos bloques están alineados, Simulink nos ofrecerá la opción de enlazarlos. Al igual que hemos hecho con los bloques, si seleccionamos un enlace podremos ponerle un nombre haciendo una doble pulsación sobre él.

El último paso que deberemos hacer en la implementación es configurar los distintos bloques para cumplir con el modelo diseñado. En nuestro ejemplo inicial hemos cambiado una de las constantes al valor predefinido. Cada bloque tiene distintas configuraciones. En este curso veremos las más importantes. Si en algún momento necesitas ayuda puedes pulsar con el botón derecho sobre un bloque y pulsar sobre la opción `help`.

6. Comentar el código. Como en todo lenguaje de programación es importante añadir comentarios sobre qué es lo que hace el programa. En el caso que sea un subsistema, explicar las variables de entrada y salida que tendremos. Podemos añadir un comentario pulsando el botón `Annotation` en el panel izquierdo o realizando una doble pulsación en la zona de diseño del modelo.

2.5 Tocando Simulink: “Primeros códigos en Simulink”

Para finalizar este tema vamos a realizar algunos modelos sencillos en Simulink para familiarizarnos con las herramientas y el uso del software.

El teorema de Pitágoras

Ejercicio 1. El primer problema que vamos a abordar es el cálculo de un cateto a partir de la hipotenusa y el otro cateto de un triángulo rectángulo. Es conocida la fórmula:

$$c_1^2 + c_2^2 = h^2.$$

Dónde c_1 y c_2 son los catetos del triángulo y h la hipotenusa (lado opuesto al ángulo recto). Por lo tanto, el cálculo que queremos desarrollar es:

$$c_1 = \sqrt{h^2 - c_2^2}.$$

Vamos a implementar este cálculo en Simulink. Para ello seguiremos los pasos siguien-

tes:

1. Creamos un nuevo modelo y lo guardamos con el nombre `cateto.slx`.
2. Abrimos el explorador de librerías (*Library Browser*).
3. Añadimos los siguientes bloques:
 - ▶ Dos bloques de entrada constante (`Source→Constant`) que representarán el cateto conocido y la hipotenusa.
 - ▶ Dos bloques de producto (`Math Operations → Product`) que usaremos para calcular los cuadrados.
 - ▶ Un bloque de resta (`Math Operations → Subtract`) para realizar la diferencia del interior de la raíz.
 - ▶ Un bloque para calcular la raíz (`Math Operations → Sqrt`).
 - ▶ Un bloque de *Display* para mostrar los resultados (`Sinks→Display`).
4. A continuación nombramos los diferentes bloques con su nombre correspondiente. Vamos a nombrar los dos bloques de entrada como c_2 y h y al bloque *Display* como c_1 . El resto de bloques tienen la función definida. Así pues, no será necesario nombrarlos. Cambiamos el valor de los bloques constantes. Al bloque h le asignamos un valor 5 y al bloque c_2 un valor de 3.
5. Unimos los bloques de forma que se calcule la fórmula deseada. Para calcular los cuadrados vamos a tener que bifurcar una misma señal. Podemos hacer esto vamos a crear un enlace que una la constante con la primera entrada del producto. Una vez creado el primer enlace realizamos la bifurcación pulsando y arrastrando con el botón derecho sobre el enlace hasta la segunda entrada. A continuación unimos la salida del cuadrado de h a la entrada de la suma del bloque *subtraction* y el cuadrado de c_2 a la entrada de la resta. La salida de este último bloque va enlazada al bloque de la raíz y finalmente del bloque de la raíz al *Display*.
6. Añadimos un comentario para recordar lo que hemos programado.

Una vez tenemos el modelo diseñado, en la Figura 13 se muestra el resultado. Podemos ejecutar el modelo pulsando sobre el botón `Run` y obtener el resultado. En este caso, el *display* nos indica que el cateto restante es 4 tal y como era de esperar.

Este modelo puede simplificarse usando el bloque de elevar al cuadrado. Lo encontraremos en `Math Operations → Math Function`. El símbolo nos indica que por

Cálculo del cateto restante a partir de la hipotenusa y un cateto

Empleamos la fórmula:

$$c_1 = \sqrt{h^2 - c_2^2}$$

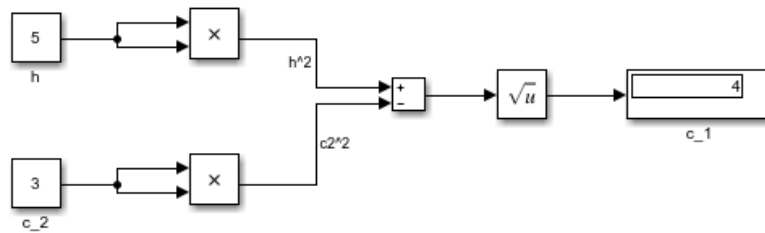


Figura 13: Obtención del cateto de un triángulo rectángulo a partir de la hipotenusa y el otro cateto.

defecto la función que calcula es una exponencial. Sin embargo, si abrimos la interfaz con una doble pulsación podremos cambiar el tipo de función a realizar. En el desplegable *Function* deberemos buscar la opción *Square*. Ahora podemos duplicar el bloque y cambiar los bloques de producto por este nuevo bloque. De forma que ahora el modelo quedará como se muestra en la Figura 14.

Cálculo del cateto restante a partir de la hipotenusa y un cateto

Empleamos la fórmula:

$$c_1 = \sqrt{h^2 - c_2^2}$$

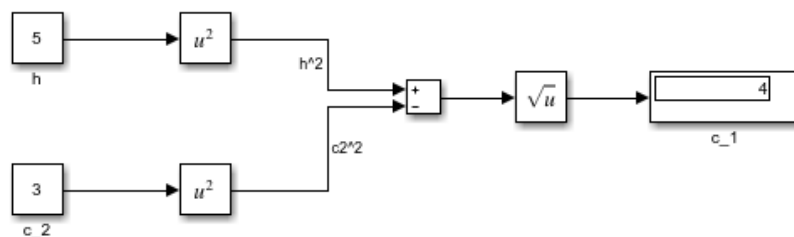


Figura 14: Obtención del cateto de un triángulo rectángulo a partir de la hipotenusa y el otro cateto usando el bloque *Square*.

Siempre que nos sea posible, intentaremos usar un bloque que minimice el número de entradas y salidas. Así el código Simulink creado será más sencillo de comprender y explicar.

En próximos temas veremos como generar subsistemas de forma que podamos emplear un código generado a modo de función.

Resolución de una ecuación de segundo grado

Ejercicio 2. En este segundo ejemplo vamos a resolver una ecuación de segundo grado a partir de los tres coeficientes a , b y c :

$$ax^2 + bx + c = 0.$$

Sabemos que la fórmula de resolución de la ecuación de segundo grado viene dada por:

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}.$$

Una vez tenemos claro el modelo a seguir vamos a implementarlo. Para ello, vamos a seguir los pasos siguientes:

1. Creamos un nuevo modelo y lo guardamos con el nombre `ecuacion2grado.slx`.
2. Abrimos el explorador de librerías (*Library Browser*).
3. Añadimos los siguientes bloques:
 - ▶ Tres bloques de entrada constante (Source→Constant) que representarán los tres coeficientes, a , b y c . Los nombramos para conocer sus valores.
 - ▶ Un bloque de cambio de signo (Math Operation → Unary Minus).
 - ▶ Un bloque de elevar al cuadrado (Math Operation → Math Function, cambiando la configuración a Square)
 - ▶ Un bloque de producto (Math Operation → Product)
 - ▶ Dos bloques de producto por escalar (Math Operation → Gain). Estos bloques sirven para multiplicar por un escalar fijo del modelo. En nuestro caso configuraremos uno a multiplicar por 4 para conseguir el $4ac$ y el otro a 2 para conseguir el $2a$ del denominador.
 - ▶ Un bloque de resta (Math Operations → Subtract) para realizar la diferencia del interior de la raíz.
 - ▶ Un bloque para calcular la raíz (Math Operations → Sqrt).
 - ▶ Un segundo bloque de resta (Math Operations → Subtract) y otro de suma (Math Operations → Add) para obtener el numerador.
 - ▶ Dos bloques de división (Math Operations → Divide) para obtener los dos resultados (el del signo positivo y el del signo negativo).



El movimiento de una bala de cañón

Ejercicio 3. Para finalizar este tema vamos a modelar el movimiento de una bala de cañón ideal (sin rozamiento y sujeta únicamente a la fuerza de la gravedad. Suponemos que el cañón dispara con un cierto ángulo θ y una velocidad v y a una altura y_0 . El objetivo será conocer la posición de la bala a lo largo del tiempo.

Para ello vamos a considerar la velocidad de salida como $(v_x, v_y) = (v \cos(\theta), v \sin(\theta))$. A continuación, definiremos la posición en la coordenada x como un movimiento rectilíneo uniforme, de forma que:

$$x = v_x t$$

Para el movimiento en la coordenada y sabemos que se trata de un movimiento uniformemente acelerado que tendrá por ecuaciones:

$$y = -gt^2 + v_y t + y_0$$

Vamos a suponer una aceleración provocada por la gravedad $g = 9.8 \text{ m} \cdot \text{s}^{-2}$. Consideraremos una velocidad inicial de $20 \text{ m} \cdot \text{s}^{-1}$ y un ángulo de disparo de $\pi/4$ y una altura de 20 m.

Una vez tenemos claro el modelo que queremos desarrollar vamos a implementarlo.

1. Creamos un nuevo modelo y lo guardamos con el nombre `bolaCanon.slx`.
2. Abrimos el explorador de librerías (*Library Browser*).
3. Añadimos los siguientes bloques:
 - ▶ Un bloque de entrada temporal (*Source→Clock*) que representará el tiempo.
 - ▶ Tres bloques de entrada constante (*Source→Constant*) para representar la velocidad inicial, el ángulo del cañón y la altura inicial. Los renombramos con sus nombres respectivos.

- ▶ Un bloque de seno (Math Operation $\rightarrow$ Trigonometric Function y otro para el coseno con el mismo bloque pero entrando en la configuración y cambiando a la opción coseno.
 - ▶ Dos bloques de producto (Math Operation $\rightarrow$ Product) para calcular el producto de la velocidad inicial por el seno y el coseno y así obtener la velocidad inicial en las dos coordenadas.
 - ▶ Un bloque de elevar al cuadrado (Math Operation $\rightarrow$ Math Function).
 - ▶ Un bloque de producto por escalar (Math Operation $\rightarrow$ Gain) para multiplicar por $-g$ el cuadrado anterior.
 - ▶ Un bloque de producto (Math Operation $\rightarrow$ Product) para realizar el producto $v_y t$.
 - ▶ Un bloque de suma (Math Operation $\rightarrow$ Add) que nos permitirá realizar la suma de los elementos. Este bloque deberemos configurarlo de forma que tenga tres entradas. Para ello, deberemos realizar una doble pulsación y en el campo list of signs poner la secuencia $+++$. Esto hará que las tres entradas sean aditivas. El número de signos nos da el número de entradas y el signo si esa entrada sumará o restará.
 - ▶ Para calcular la posición en x necesitaremos únicamente un bloque de producto (Math Operation $\rightarrow$ Product).
 - ▶ En esta ocasión, vamos a añadir un bloque de gráfico en coordenadas x e y (Sinks $\rightarrow$ XY Graph) que nos permitirá obtener un gráfico con las entradas correspondientes. Cuando introducimos un nuevo bloque de gráficos debemos configurar sus límites de visualización. Para ello vamos a realizar una doble pulsación sobre el elemento y fijar los límites de la siguiente forma. El límite inferior en x a -1 , el límite superior a 40 , el límite inferior en y a -1 y el superior a 30 .
4. Asignamos los valores de entrada a las constantes $\theta = \pi/4$, $v = 20$ e $y = 20$.
5. Unimos los bloques de forma que se calcule la fórmula deseada. Deberemos unir la constante θ a las entradas de los bloques seno y coseno. A continuación uniremos el bloque constante v a la primera entrada de los primeros bloques de producto. La segunda entrada deberá recibir la salida de las funciones seno y coseno respectivamente. Con esto habremos completado las velocidades iniciales en las coordenadas x e y . Para crear la evolución de la coordenada x deberemos conectar la salida del producto asociado al coseno (que nos da v_x) y la salida del bloque de tiempo. Para la coordenada y deberemos conectar el reloj al bloque que nos da el cuadrado, y conectar éste al bloque del producto por escalar para obtener el término cuadrático. Para el término lineal deberemos multiplicar el bloque del producto que nos

da la velocidad inicial en y por el tiempo. Finalmente conectaremos el bloque del término lineal, el del término cuadrático y la altura inicial al bloque de la suma con tres entradas. Para terminar las conexiones conectaremos las posiciones de cada bloque a la salida gráfica.

6. Añadimos un comentario para recordar lo que hemos programado. También añadimos comentarios sobre los enlaces para clarificar los datos y señales de nuestro modelo

Cuando ejecutamos este modelo obtenemos una solución que sobrepasa el límite del suelo. En temas posteriores vamos a ver estrategias para poder valorar el momento en el que parar la simulación acorde a alguna de las variables. La Figura 16 muestra el modelo que acabamos de desarrollar junto con el gráfico generado por la simulación.

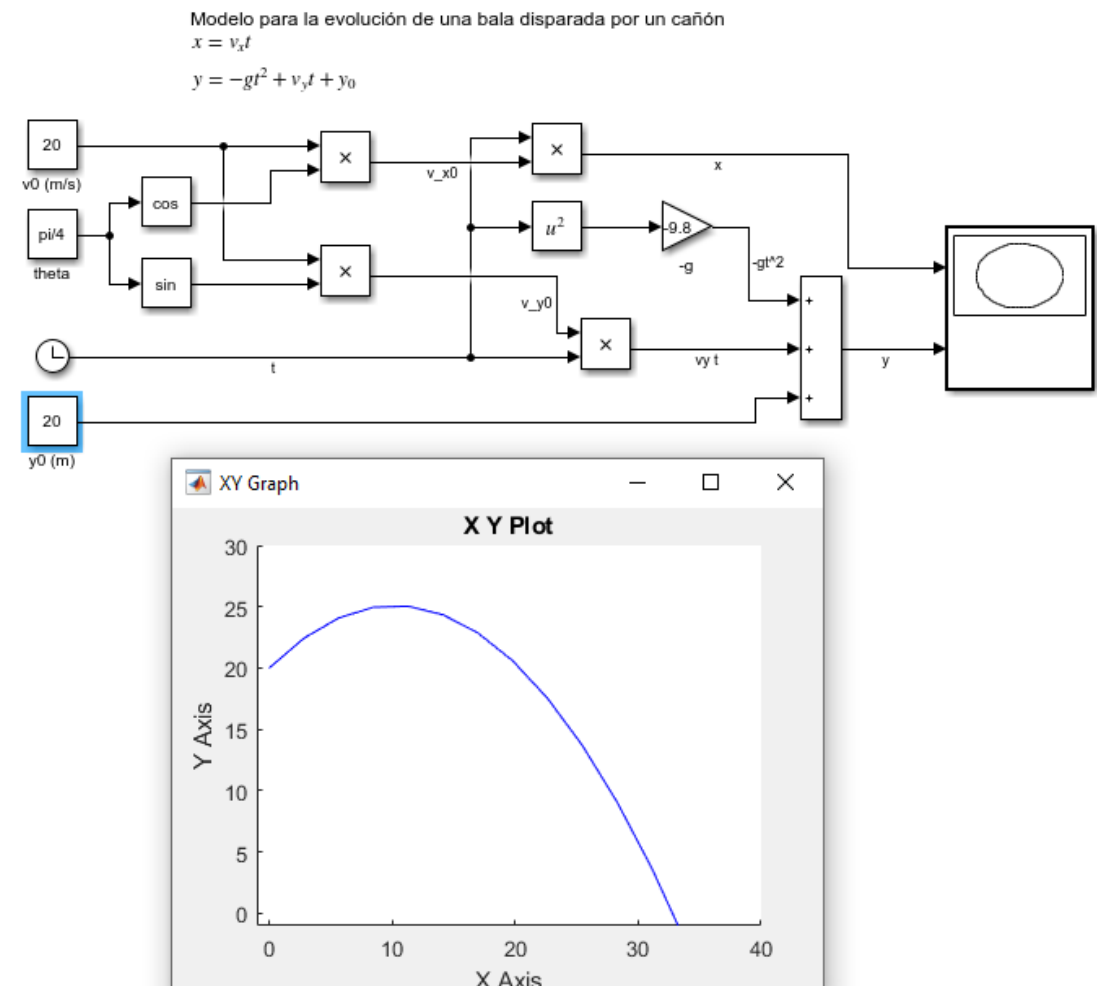


Figura 16: Modelo de simulink para obtener la evolución temporal de una bala de cañón disparada desde una altura de 20 metros con un ángulo de $\pi/4$ y una velocidad inicial de 20m/s.